

COMS10015: Revision Questions

2025-26

SOLUTIONS

Contents

1	Defintions	2
2	Architecture and Micro-architecture	4
3	ALU	12
4	Memory	17
5	Hardware-support for Software	23
6	Assembly and Assemblers	30
7	Compilers	35

1 Definitions

1. Define or explain the following terms:

1. Micro-architecture
2. Instruction Set Architecture (ISA)
3. Opcode
4. Operand
5. ISA addressing modes
6. Data Path
7. Control Path
8. Decode
9. Control
10. 2-phase clock
11. Hardwired control
12. Micro-programmed control
13. Micro-instructions
14. Micro-program
15. CISC and RISC
16. Speculative execution
17. Bus, and their properties (serial/parallel, synchronous/asynchronous)
18. Arithmetic logic unit (ALU)
19. Ripple-carry full adder
20. Adder-subtractor
21. Two's complement overflow
22. Status registers
23. Special purpose registers
24. General purpose registers
25. Wallace Tree
26. Same Instruction, Multiple Data (SIMD)
27. Execution cycle
28. Fetch/Decode/Execute/Writeback
29. Instruction Pipelining
30. Bubbles
31. Cycles per Instruction (CPI)
32. Iron Law of processor performance
33. Hazards (data, structural, and control)

34. Data dependencies (RAW, WAR, and WAW)
35. Memory hierarchy
36. von Neumann architecture
37. Cache inclusivity
38. Caches (instruction and data)
39. Working set
40. Cache line
41. Cache hits and misses
42. Locality (temporal and spatial)
43. Direct mapped cache
44. Cache eviction
45. Tag Store
46. Valid bit
47. Stack
48. Stack pointer
49. Application binary interface and the mechanism for subroutine calls
50. Stack overflow
51. Buffer overflow
52. Return address
53. Stack frame
54. Interrupt
55. Exception
56. Interrupt handler
57. Interrupt vector
58. Program counter
59. System calls
60. Software interrupts
61. Timing interrupt
62. Protection
63. Privileged execution
64. Position independent code
65. Virtual memory
66. Pages
67. Virtual addresses
68. Physical addresses
69. Page table
70. Page faults
71. Translation Lookaside Buffer (TLB)

2 Architecture and Micro-architecture

2. The Hex 8 ISA has 8-bit instructions consisting of a 4-bit opcode and a 4-bit operand. The PFX (prefix) instruction allows the following:
- A. System calls.
 - B. Memory protection.
 - C. 8-bit operands.**
 - D. 8-bit opcodes.
 - E. The processor to halt execution.

Solution: The PFX instruction comes from the Transputer. It combines its 4-bit operand with the 4-bit operand of the following instruction. The PFX operand is used as the most-significant bits of the 8-bit operand of the following instruction.

3. The Hex 8 ISA has 8-bit instructions consisting of a 4-bit opcode and a 4-bit operand. The PFX (prefix) instruction allows operands to be extended to 8-bits, at the cost of executing this extra instruction.

Which of the following changes would allow for 8-bit operands, without the need for an extra instruction or increasing the latency of the instructions?

- A. Extend the instruction length to 12 or more bits.**
- B. Store operands in memory.
- C. Store operands in a register.
- D. Use an assembler to extend the operands by 4 bits.
- E. Append a fixed 4-bit number to the operands.

Solution: The PFX instruction comes from the Transputer. It combines its 4-bit operand with the 4-bit operand of the following instruction.

The question asks what other solution could extend the operands, without increasing the number of instructions or adding latency. The only viable solution of those given is to extend the length of the instruction.

The other answers would either not work (appending a fixed 4-bit number or use a “magic” assembler), or increase latency and/or require additional instructions to support 8-bit operands (storing in registers/memory).

4. The Hex 8 ISA has a word length of 8 bits. Each instruction is formed of a 4-bit opcode and a 4-bit operand, which limits the number of instructions to 16.

Some of the instructions in the Hex 8 ISA do not require operands. Which of the following schemes would allow the ISA to be expanded to contain 31 instructions, while retaining 4-bit opcodes which can be decoded in a single cycle?

- A. **Include an instruction to specify the operand is to be interpreted as an opcode of an instruction which doesn't require an operand. E.g. EXE ADD**
- B. Send an interrupt to indicate the instruction opcode is larger than 16. E.g. A 4-bit opcode refers to two instructions depending on an interrupt signal
- C. Use a mechanism similar to the PFIX instruction to construct the opcode from two instructions
- D. Write a micro-program to implement the additional instructions

Solution: The only way to get 31 instructions of the solutions listed is to use one instruction of the original 16 to designate the operand is the opcode. This yields $15 + 16 = 31$ instructions.

The other methods yield 32 and are impractical, require two cycles or are incorrect (respectively).

5. Many Instruction Set Architectures, including x86 and Arm, provide extensions to support SIMD vector instructions. These instructions perform the same operation on each element of the vector.

Two input programs are written, both performing the same total number of computations (arithmetic operations). One uses SIMD vector instructions and the other does not.

Which of these statements is true?

- A. **The program which uses vector instructions results in fewer instructions to be fetched and decoded.**
- B. The program which uses vector instructions results in more instructions to be fetched and decoded.
- C. The program which uses vector instructions results in more computations to be performed by the processor.
- D. The program which uses vector instructions results in fewer computations to be performed by the processor.
- E. The program which uses vector instructions results will always take longer to execute.

Solution:

The vector program will contain fewer instructions, and so fewer instructions will be fetched/decoded for a fixed amount of arithmetic.

The question contains no detail about the speed of execution or the implementation of the SIMD instructions in the micro-architecture. Therefore the answer regarding speed of execution is not always true.

6. When a micro-architecture is designed, which of the following factors might be considered when implementing a particular architecture?

- A. Thermal design power (the maximum heat generated by the processor).
- B. The speed of the clock.
- C. Production yield.
- D. All of the above.**
- E. None of the above.

Solution: All of the three listed factors are important when implementing an architecture, along with many others.

7. A processor uses a micro-programmed control unit. There are many advantages to this approach over a hardwired control unit. Which one of these statements correctly describes such an advantage?
- A. It can support more instructions than a hardwired control unit
 - B. It allows a more complex instruction set to be implemented more conveniently**
 - C. General programs can change the micro-code so users can change the processor function
 - D. Micro-programs are the only way to implement a control unit

Solution: Application of knowledge from the lectures.

Micro-programs are hidden from the ISA so cannot be changed by the users. Hardwired control is an alternative implementation of the control unit which can implement lots of complex instructions; although in practice it wouldn't be done it is still possible.

The correct answer is therefore that micro-programmed control a matter of convenience.

8. A Control Unit in the micro-architectural design of a processor is used to control the Data Path so that it performs whatever is required for the current instruction. Some processor designs implement this unit manually, but others use micro-programs instead, especially when there are a large number of instructions. When an instruction is decoded, what must the Decoder do in a micro-programmed control processor?
- A. Fetch the next instruction from memory.
 - B. Set the architectural program counter according to the decoded opcode.
 - C. Set the micro-program counter to the start of the corresponding micro-program for the decoded opcode.**
 - D. Send a 1-bit signal on the control bus to the Control Unit corresponding to the decoded opcode.
 - E. Put the opcode on the data bus.

Solution: The decoder will set the micro-program counter to the address of the micro-program for that particular opcode.

The main program counter won't be set as a result of the decoder (ever) – branch instructions may do this, but not directly by the decoder.

A 1-bit signal on the control bus will be insufficient for an ISA with more than 2 instructions. The question highlighted the case where there are a large number of instructions. Selecting this answer would suggest a hardwired control unit, but the question says the processor is controlled by a micro-program.

The other options are spurious.

9. An Instruction Set Architecture (ISA), such as Arm or x86, gives lots of information about what a processor should do. Which of the following does an ISA *not* define?
- A. The number of architectural registers.
 - B. The latency of each instruction.**
 - C. The instruction format.
 - D. The memory addressing modes.
 - E. Binary representation of the opcodes.

Solution: The ISA defines all things listed except the latency of each instruction, which is a micro-architectural feature.

It might be possible to implement an ISA with a different binary representation of each opcode, however this would result in binary incompatibility of programs conforming to the ISA, and this is not done in practice.

10. A processor designer has correctly implemented an ISA, and used a micro-programmed control unit. They have only provided one program counter register for use by the stored program and the control micro-program. What is the problem with this?
- A. The micro-program will overwrite memory.
 - B. Program instructions may be skipped.
 - C. The implementation is inefficient as the program counter must be saved to memory so that it can be used to execute the micro-program for each instruction.**
 - D. The micro-program is incorrect.
 - E. The program must be written in micro-instructions.

Solution: The micro-programmed control unit typically has its own micro-program counter so that it can execute the micro-program. With a shared register, the (main) program counter needs to be backed up each cycles, which is inefficient. Without

doing this, program instructions could be skipped, however the question assumes the implementation is correct. Writing programs directly in micro-instructions would resolve the issue of a lack of a separate program counter, however would no longer be valid on the processor as it implements the ISA, not the micro-program.

11. A program has the following mix of instructions:

Instruction	Cycles	Frequency
Load	8	30%
Store	9	15%
Branch	11	25%
Arithmetic	3	30%

What is the average number of clock cycles per instruction (CPI) for this program?

- A. 1.00
- B. 7.40**
- C. 7.75
- D. 23.87
- E. 740.00

Solution: The CPI is computed as the weighted average of the instruction cycles where the weights are the frequencies.

$$CPI = \frac{8 \times 30 + 9 \times 15 + 11 \times 25 + 3 \times 30}{100} = 7.40$$

12. A program has the following mix of instructions:

Instruction	Cycles	Frequency
Load	4	20%
Store	4	15%
Branch	8	5%
Arithmetic	1	60%

What is the average number of clock cycles per instruction (CPI) for this program?

- A. 2.40**
- B. 14.12
- C. 2.08
- D. 43.53
- E. 0.17

Solution: The CPI is computed as the weighted average of the instruction cycles where the weights are the frequencies.

$$CPI = \frac{4 \times 20 + 4 \times 15 + 8 \times 5 + 1 \times 60}{100} = 2.4$$

13. A breakdown of the instructions in a program is given in the following table. The table gives the number of cycles taken by each instruction.

Instruction	Cycles	Count
Load	4	20
Store	4	15
Branch	8	5
Arithmetic	1	58

Consider a scalar (non-pipelined) processor which has no data caches, clocked at 1000 cycles per second (1 KHz). How long does the program in the above table take to run on this processor?

- A. 0.238 seconds.
- B. 2.380 seconds.
- C. 2.429 seconds.
- D. 0.243 seconds.
- E. 1.666 seconds.

Solution:

As a scalar processor, the instructions will be executed sequentially, taking their full time with no overlap (which would be the case for pipelining). Therefore the total number of cycles is the sum of the products of instruction counts and their latencies. The program takes $4 \times 20 + 4 \times 15 + 8 \times 5 + 1 \times 58 = 238$ cycles.

The processor is 1 KHz, processing 1000 cycles per second, so the time taken for execute the program is $238/1000 = 0.238$ seconds.

A common mistake might be to attempt to compute the CPI, as the weighted average of the data in the table. The instruction counts deliberately do not sum to 100 to try to hint this is the wrong approach. The second answer is the weighted average, and the third is the CPI calculated correctly however both are the wrong answer for the question.

14. A scalar pipelined processor has a 4-stage execution cycle: Fetch – Decode – Execute – Write. What is the theoretical maximum throughput (the number of completed instructions per cycle) of the processor?
- A. 4 instructions per cycle.

B. 1 instruction per cycle.

- C. It depends on the number of bubbles/stalls.
- D. It depends on the instruction latency.
- E. It depends on the clock speed.

Solution: This is a tricky question, although just requires synthesis of knowledge from the lectures. The lecture slides do show this as an example, although it is a complicated idea, and required deep understanding to reconstruct in this closed-book exam.

A scalar pipelined processor can at retire *at most* one instruction each cycle.

The 4-stage execution cycle means at most 4 instructions are in flight (one in each stage), but the question asks for throughput as defined in completed instructions per cycle.

Instructions with long latency will cause bubbles in the pipeline, and therefore more *cycles per instruction* will be required. This CPI metric is shown in lectures but is the inverse of the question here, which asks for a maximum number of instructions per cycle, all other things being equal.

15. What type of data dependency hazard occurs in following instruction stream?

$$r1 \leftarrow r2 + r3$$

$$r4 \leftarrow r3 + r4$$

$$r5 \leftarrow r3 + r1$$

A. Read after Write (RAW)

- B. Read after Read (RAR)
- C. Write after Write (WAW)
- D. Write after Read (WAR)

Solution: The register $r1$ is read on line 3 after being written on line 1, therefore the answer is Read after Write (RAW).

Line 2 is not a hazard because although $r2$ is read and written, it is within the same instruction and therefore does not impose a data dependence.

16. What type of data dependency hazard occurs in following instruction stream?

$$r0 \leftarrow r2 + r3$$

$$r1 \leftarrow r3 + r4$$

$$r5 \leftarrow r3 + r1$$

A. Read after Write (RAW)

B. Read after Read (RAR)

C. Write after Read (WAR)

D. Write after Write (WAW)

Solution: The register $r1$ is read on line 3 after being written on line 1 and 2, therefore the answer is Read after Write (RAW).

17. What type of hazard is likely to occur in following instruction stream when it is executed on a pipelined processor with three arithmetic logic units (ALUs)?

$$r1 \leftarrow r2 + r3$$

$$r4 \leftarrow r5 + r4$$

$$r1 \leftarrow r2 + r6$$

$$r7 \leftarrow r3 + r5$$

A. Read after Read (RAR) data hazard.

B. Write after Read (WAR) data hazard.

C. Read after Write (RAW) data hazard.

D. Write after Write (WAW) data hazard.

E. Control hazard.

Solution: The register $r1$ is written on lines 1 and 3, therefore there is could be a Write after Write (WAW) data hazard.

Line 2 is not a hazard because although $r4$ is read and written, it is within the same instruction and therefore does not impose a data dependence.

There are no branch instructions so no opportunities for a control hazard. Read after Read (RAR) is made up.

3 ALU

18. Figure 1 shows a circuit for 5-bit multiplication, which is constructed with half adders (HA) and full adders (FA). Why is this circuit *not* a Wallace Tree?
- A. It computes the addition of two numbers instead.
 - B. The layers use half adders when they should use full adders.
 - C. Wallace Trees only ever have three layers.
 - D. The layers use full adders when they should use half adders.
 - E. The layers do not follow the Wallace Tree procedure of combining 2 or more wires with the appropriate adder.**

Solution: Figure 1 is a shift-adder circuit for multiplication, so is *not* a Wallace Tree. Specifically, it doesn't follow the procedure for adding layers.

19. A ripple-carry full adder can be turned into an adder-subtractor by converting one of the inputs to two's complement. The control signal is sent to the first carry-in bit of the first 1-bit full-adder. Which logic gate can be used to combine each bit of one of the inputs with the control signal to complete the conversion in order to compute subtraction?
- A. AND gate.
 - B. OR gate.
 - C. XOR gate.**
 - D. NOT gate.
 - E. NAND gate.

Solution: One input needs to be inverted with a logical NOT operation only if the control signal is active.

If a NOT gate is used, a multiplexor needs to select between the two versions; the question asks for the input to be combined with the control signal, and so a NOT gate alone doesn't combine the signals.

AND, OR and NAND gates have the incorrect logic tables for the required inversion of the input bit based on the control bit.

The XOR gate provides the correct inversion.

Question 20 and Question 21 both refer to the two's complement addition of the 8-bit numbers 00011100 (28 in decimal) and 01100011 (99 in decimal).

20. Compute the two's complement addition of the two 8-bit numbers.

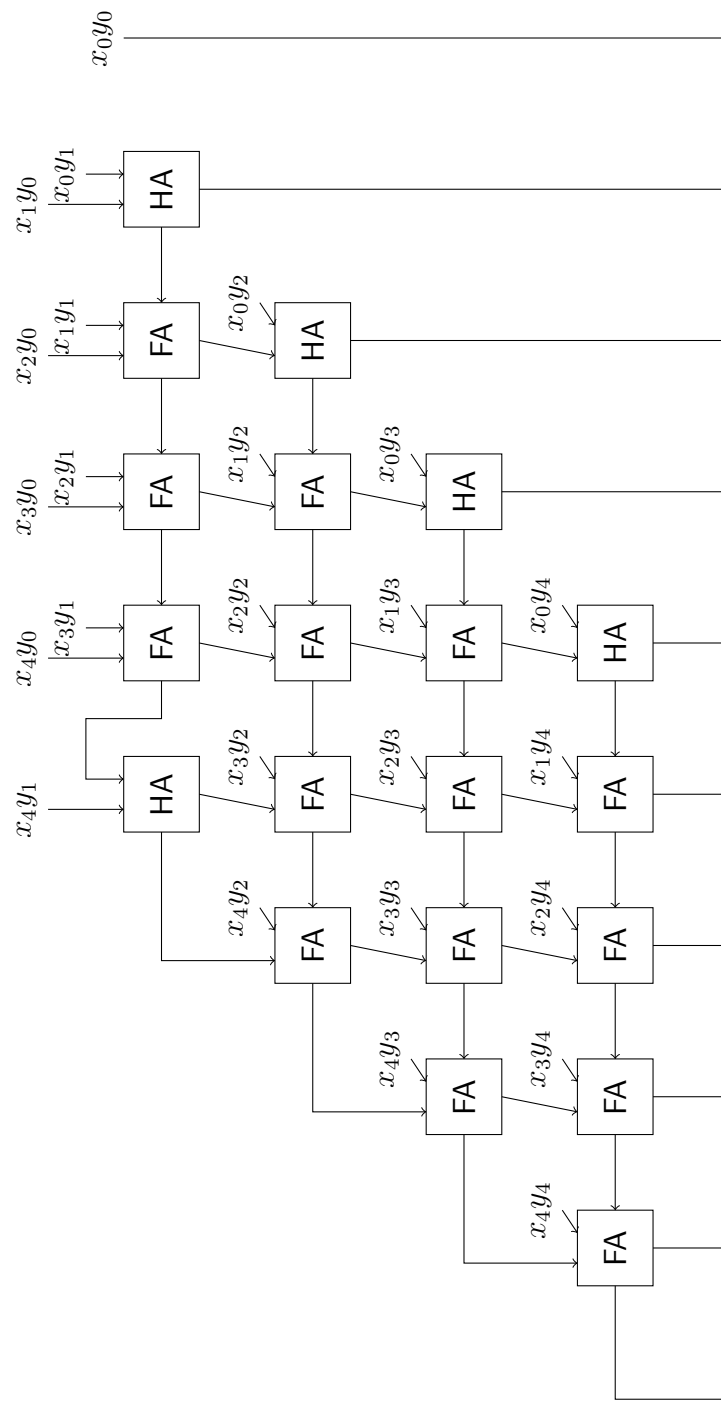


Figure 1: A 5-bit multiplication circuit.

- A. 01000111 (71 in decimal)
- B. 10111001 (-71 in decimal)
- C. 10000001 (-127 in decimal)
- D. 01111111 (127 in decimal)**

Solution:

```

00011100 (28 in decimal)
01100011 (99 in decimal)
-----
01111111 result
00000000 carry out

```

21. Did the result overflow?

- A. Yes
- B. No**

Solution: The two most significant carry out bits are the same, therefore overflow *has not* occurred.

An alternative explanation is in Two's Complement, the addition to the two positive numbers yielded a positive number, by observation, although the mechanism for checking this in hardware is by inspection of the carry bits.

22. Compute the two's complement addition of the following two 8-bit numbers:

- 01100100
- 10111000

The result is:

- A. 10101100 (-84 in decimal).
- B. 11011100 (-36 in decimal).
- C. 00011100 (28 in decimal).**
- D. 11100100 (-28 in decimal).
- E. 11111100 (-4 in decimal).

Solution: This question asks to sum 100 and -72.

```

01100100
10111000
-----
00011100 result
11100000 carry out

```

Question 23 and Question 24 both refer to the two's complement addition of the 8-bit numbers 11011101 and 00100101.

23. Compute the two's complement addition of the two 8-bit numbers.
- A. 11111000 (-8 in decimal).
 - B. 11111111 (-1 in decimal).
 - C. 00000010 (2 in decimal).**
 - D. 00000101 (5 in decimal).
 - E. 01001000 (72 in decimal).

Solution: This question asks to sum -35 and 37.

```

11011101
00100101
-----
00000010 result
11111101 carry out

```

24. Did the result overflow?
- A. Yes.
 - B. No.**

Solution: The question uses two's complement arithmetic, so the final two carry bits need checking. The final two carry bits are the same (1), therefore overflow *has not* occurred.

25. Compute the two's complement addition of the following two 8-bit numbers:
- 01100011
 - 11001000

Does the result cause an overflow, and why?

- A. Yes, there was an overflow because the final carry bit is one.
- B. Yes, there was an overflow because the final two carry bits are different.
- C. No, there was not an overflow because the final two carry bits are different.
- D. Yes, there was an overflow because the final two carry bits are the same.
- E. No, there was not an overflow because the final two carry bits are the same.**

Solution: This question asks to sum 99 and -56, to equal 43.

```
01100011
11001000
-----
00101011 result
11000000 carry out
```

There is not overflow because the final two carry out bits match. Note the carry out on the final (left-most) binary addition is 1 because the carry is carried.

26. Compute the two's complement addition of the following two 8-bit numbers:

- 11101011
- 11011010

Does the result cause an overflow, and why?

- A. No, there was not an overflow because the final two carry bits are different.
- B. No, there was not an overflow because the final two carry bits are the same.**
- C. Yes, there was an overflow because the final two carry bits are different.
- D. Yes, there was an overflow because the final two carry bits are the same.
- E. Yes, there was an overflow because the final carry bit is one.

Solution: This question asks to sum -21 and -38, to equal -59.

```
11101011
11011010
-----
11000101 result
11111010 carry out
```

There is not overflow because the final two carry out bits match.

4 Memory

27. Explain the key features of the following memory technologies. You should discuss the performance, volatility, and capacity.

1. SRAM
2. DRAM
3. HBM
4. Flash
5. Hard disk
6. Tape

Solution:

SRAM Volatile memory made of flip-flops. Made just of logical components, it is the fastest type of memory, and as such tends to have limited capacity in part because it is not that dense in terms of the physical size of each component. It is mainly used for caches.

DRAM Volatile memory used as the main memory in processors. Constructed using logic and capacitors, with the capacitor being used to store charge, representing a binary value. It needs regular refresh to ensure that the data values are maintained in the capacitors.

HBM High-bandwidth memory. A 3D stacked memory technology used to provide high bandwidth main memory. It has higher bandwidth than DRAM, and slightly higher latency. It is volatile.

Flash High capacity, non-volatile storage. Often used as the main persistent data store in commodity hardware, or for fast storage in large-scale data centres. Flash is relatively fast to access compared to the other non-volatile storage technologies such as HDD and tape.

Hard disk High capacity, non-volatile storage. Often used for bulk storage in desktop and workstation machines, although flash is often used in more portable devices.

Tape Very high capacity, non-volatile storage, often used for archival purposes. Very slow to access random data due to linear nature of the tape.

28. Which type of locality does the loop in the following C program demonstrate?

```
float A[N];
float sum = 0.0;
for (int i = 0; i < N; ++i) {
    sum += A[i] + A[i+1];
}
```

- A. Just temporal locality.
- B. Just spatial locality.
- C. Both temporal and spatial locality.**
- D. Neither; there is no locality in this loop.

Solution: Both spatial locality and temporal locality are observed in the loop. When we access $A[i]$ we also access data in the nearby location $A[i+1]$. That data is also reused in the following iteration of the loop, resulting in temporal locality.

29. In a given computer system, accesses to main memory by the processor are supported by a 16 KiB ($16\,384 = 2^{14}$ bytes) direct mapped cache. Memory addresses are 16 bits, and each addressable element has a word size of 4 bytes. Cache blocks are of size 64 bytes. Locations in the cache are numbered starting at 0.

Consider the memory address 0101010110101010 (decimal 21930) where the least-significant bit is on the right-hand side.

Which location in cache is data stored at this address placed, and what is the value in the Tag store?

- A. Location 01011010 (decimal 90), Tag store 0101.**
- B. Location 10101010 (decimal 170), Tag store 01010101.
- C. Location 1010 (decimal 10), Tag store 01010101.
- D. Location 01011010 (decimal 90), Tag store 1010.
- E. Location 10101010 (decimal 170), Tag store 0101.

Solution: Cache blocks are of size 2^6 bytes, but the memory is addressable in words of 4 bytes. Therefore, there are $2^6/2^2 = 2^4$ words per cache line; hence the lowest 4 bits of the memory address are used to index within a cache line. Note, the lowest 4 bits of the memory address are safe to ignore in this question.

There are $2^{14}/64 = 2^{14}/2^6 = 2^8$ locations in cache. Therefore, 8 bits of the memory address are used to determine the location. They are the 5th–12th least-significant bits, inclusive.

Finally, the remaining 4 most-significant bits are stored in the Tag.

30. In a given computer system, accesses to main memory by the processor are supported by a 32 KiB ($32\,768 = 2^{15}$ bytes) direct mapped cache. Memory addresses are 16 bits, and each addressable element has a word size of 4 bytes. Cache blocks are of size 64 bytes. Locations in the cache are numbered starting at 0.

Consider the memory address 1101110001101011 (decimal 56427) where the least-significant bit is on the right-hand side.

Which location in cache is data stored at this address placed, and what is the value in the Tag store?

- A. Location 001101011 (decimal 107), Tag store 1101110.
- B. Location 111000110 (decimal 454), Tag store 110.**
- C. Location 1011 (decimal 11), Tag store 11011100.
- D. Location 110001101 (decimal 397), Tag store 011.
- E. Location 110111000 (decimal 440), Tag store 1101011.

Solution: Cache blocks are of size 2^6 bytes, but the memory is addressable in words of 4 bytes. Therefore, there are $2^6/2^2 = 2^4$ words per cache line; hence the lowest 4 bits of the memory address are used to index within a cache line. Note, the lowest 4 bits of the memory address are safe to ignore in this question.

There are $2^{15}/64 = 2^{15}/2^6 = 2^9$ locations in cache. Therefore, 9 bits of the memory address are used to determine the location. They are the 5th–13th least-significant bits, inclusive.

Finally, the remaining 3 most-significant bits are stored in the Tag.

31. In a given computer system, accesses to main memory by the processor are supported by a 16 KiB ($16\,384 = 2^{14}$ bytes) direct mapped cache. Memory addresses are 16 bits, and each addressable element has a word size of 1 byte. Cache blocks are of size 64 bytes. Locations in the cache are numbered starting at 0.

Consider the memory address 1010 0110 1001 1111 where the least-significant bit is on the right-hand side.

Which location in cache is data stored at this address placed, and what is the value in the Tag store?

- A. Location 1001 1111, Tag store 1010 0110.
- B. Location 1010 0111, Tag store 11.
- C. Location 10 0110 1001 1111, Tag store 10.
- D. Location 1001 1010, Tag store 10.**
- E. Location 10 1001 1010 0111, Tag store 11.

Solution: Cache blocks are of size $64 = 2^6$ bytes, but the memory is addressable in words of 1 bytes. Therefore, there are $2^6/2^0 = 2^6 = 64$ words per cache line; hence the lowest 6 bits of the memory address are used to index within a cache line. As such, the lowest 6 bits of the memory address are safe to ignore in this question.

There are $2^{14}/64 = 2^{14}/2^6 = 2^8$ locations in cache. Therefore, 8 bits of the memory address are used to determine the location. They are the 7th–14th least-significant bits, inclusive.

Finally, the remaining 2 most-significant bits are stored in the Tag.

32. A processor has a memory hierarchy containing:

- Main memory.
- L3 exclusive cache.
- L2 inclusive cache.
- L1 cache.
- Registers.

A cache line is present in the L1 cache. In which other caches can this cache line be guaranteed to be found?

- A. No other caches; the data is only in the L1 cache.
- B. L2; the data can be found in the L1 and L2 caches.**
- C. L2 and L3; the data can be found in all three caches (L1, L2 and L3).
- D. L3; the data can be found in the L1 and L3 caches.
- E. The data is tagged as evicted from the L1 cache.

Solution: The L2 cache is inclusive of L1, meaning it contains an entire copy of the L1 cache, plus extra cache lines. Therefore, as the data is in L1, it *will also* be found in L2.

The L3 cache is exclusive, meaning it only contains data evicted from L2 cache. Therefore, the cache line is not guaranteed to be present there.

33. A computer system supports up to 2^{36} bytes (64 GiB) of byte-addressable virtual memory. There are 2^{30} bytes (1 GiB) of physical memory installed. The memory is split in pages of size 4 KiB (2^{12} bytes).

How large, in MiB (mebibytes), is the Page Table?

- A. 36 MiB.**
- B. 0.75 MiB.
- C. 24 MiB.
- D. 288 MiB.
- E. 3 MiB.

You may wish to first calculate the number of entries in the Page Table, and the size of each entry.

Recall:

- There are 8 bits per byte.
- There are 1024 bytes in a KiB.
- There are 1024 KiB in a MiB.
- There are 1024 MiB in a GiB.

Solution: There are $2^{36-12} = 2^{24}$ pages, and so the Page Table needs this many entries.

We can store $2^{30-12} = 2^{18}$ pages in physical memory, and so each entry of the Page Table stores 18 bits to identify the physical addresses.

The page table is therefore $2^{24} \times 18$ bits, or $(2^{24} \times 18)/(8 \times 1024 \times 1024) = 36$ MiB.

34. The Translation Lookaside Buffer (TLB) stores which of the following:

- A. Physical memory addresses.**
- B. Virtual memory addresses.
- C. A list of all pages.
- D. A cache of recently accessed data.
- E. Decoded instructions.

Solution: TLBs are an optimisation for virtual to physical memory address translation. Therefore, the cache stores the physical addresses. The virtual addresses are used as an index/key to the cache.

Note, the TLB doesn't enumerate all the pages: that is the job of the Page Table.

35. A particular computer architecture supports up to 2^{30} bytes (1 GiB) of byte-addressable virtual memory. The ISA for this architecture therefore uses 30-bit memory addresses.

There are 2^{28} bytes (256 MiB) of physical memory installed. The memory is split in pages of size 1 KiB (2^{10} bytes).

A Translation Lookaside Buffer (TLB) is used to speed up the translation of virtual memory addresses to physical memory addresses.

How many bits of the address need to be stored for each entry in the TLB?

- A. 2 bits.
- B. 10 bits.
- C. 18 bits.**
- D. 20 bits.
- E. 30 bits.

Solution: The system can store $2^{28-10} = 2^{18}$ pages in physical memory. Therefore each entry of the Page Table, and therefore the TLB, stores 18 bits to identify the physical addresses.

The other answers could be derived by following the *incorrect*, but related, ideas:

- There are $2^{30-10} = 2^{20}$ (virtual) pages, so the Page Table needs this many entries.
- By the above, 20 bits are used as an index to the Page Table, so $30 - 20 = 10$ bits are kept (passed through unmodified) in the translated physical address. The other 18 bits come from the translated physical address, stored in the Page Table/TLB.
- Memory address are 30-bit, but the Page Table and TLB do not store the full virtual address.
- 2 bits is the result of incorrectly calculating the ratio of addressable virtual memory to physical memory.

36. A computer system supports up to 2^{32} bytes (4 GiB) of byte-addressable virtual memory. There are 2^{27} bytes (128 MiB) of physical memory installed. The memory is split in pages of size 16 KiB (2^{14} bytes).

What is the minimum size of the Page Table, in KiB (kibibytes)?

- A. 5 KiB.
- B. 8 KiB.
- C. 18 KiB.
- D. 416 KiB.**
- E. 576 KiB.

You may wish to first calculate the number of entries in the Page Table, and the size of each entry.

Recall:

- There are 8 bits per byte.
- There are 1024 bytes in a KiB.
- There are 1024 KiB in a MiB.
- There are 1024 MiB in a GiB.

Solution: There are $2^{32-14} = 2^{18}$ pages, and so the Page Table needs this many entries.

We can store $2^{27-14} = 2^{13}$ pages in physical memory, and so each entry of the Page Table stores 13 bits to identify the physical addresses of each page.

The page table is therefore $2^{18} \times 13$ bits, or $(2^{18} \times 13)/(8 \times 1024) = 416$ KiB.

5 Hardware-support for Software

37. Programs use stacks in memory to support subroutine calls. Typically the stacks are descending, where pushing data onto the stack moves the stack pointer to a lower address. Why is a descending stack more convenient than an ascending stack?

- A. Implementing an addition circuit is easier than a subtraction one
- B. Ascending stacks could run out of memory and descending stacks cannot
- C. Subroutine cannot access calling parameters in the stack frame
- D. The total amount of stack memory required by a program is unknown at compile time, so by starting at high addresses the stack has the maximum amount of room to expand**

Solution: This is application of knowledge of stacks by thinking over the consequences of the statements.

The students know how to implement an adder-subtractor so the computation of offsets is easy in both cases.

Both ascending and descending stacks can run out of memory if they get too large.

Subroutines can access parameters in both cases.

The correct answer is the stack size is unknown in advance, so descending give the largest amount of space safely.

38. A stack data structure is often used to support subroutine calling in a high level language like C.

Who or what is responsible for allocating enough space on the stack for a subroutine's local variables?

- A. The programmer.
- B. The compiler.**
- C. The loader.
- D. The linker.
- E. The special registers.

Solution: Tests understanding of the subroutine calling process.

The compiler will insert instructions to decrease the stack pointer sufficiently for subroutine's local variables.

The programmer might have to do it by hand when writing assembly, but this question specifies a high level language.

39. Why is it important to support different execution privileges at an architectural level?

- A. To ensure user programs cannot access restricted portions of memory
- B. To ensure kernel cannot access application memory
- C. To allow relocation of programs in memory
- D. To allow the kernel to schedule multiple programs on a single processor

Solution: The architecture must put in place some restrictions to ensure that load/store instructions to certain memory addresses will only succeed based on the privilege level. For example, a load/store from kernel code should always work, but a load/store in user applications should only succeed if the memory address is within the allowable range, or if they are accessing the page table for virtual-to-physical address translation, etc.

40. In adding protection to the Hex 8 ISA, we must ensure that only the kernel can access all memory addresses. To do this, we add a flag *ink* which is 1 if the processor is executing instructions associated with the kernel. We also add two registers: *ab*, the application base memory address; and *as*, the application size.

The LDBI instruction was originally defined as follows:

LDBI $breg \leftarrow mem[breg + oreg]$

Which of the following is the correct definition of a *protected* LDBI Hex 8 instruction?

- A. **if** *ink* **is** 1 **then** $breg \leftarrow mem[breg + oreg]$
else if $(breg + oreg) < as$ **then** $breg \leftarrow mem[ab + breg + oreg]$
else error
- B. **if** *ink* **is** 0 **then** $breg \leftarrow mem[breg + oreg]$
else if $(breg + oreg) < as$ **then** $breg \leftarrow mem[ab + breg + oreg]$
else error
- C. **if** *ink* **is** 1 **then** $breg \leftarrow mem[breg + oreg]$
else if $(breg + oreg) < as$ **then** $breg \leftarrow mem[breg + oreg]$
else error
- D. **if** *ink* **is** 1 **then** $breg \leftarrow mem[breg + oreg]$
else if $(breg + oreg) < as$ **then** $breg \leftarrow mem[as + breg + oreg]$
else error
- E. **if** *ink* **is** 1 **then** $breg \leftarrow mem[breg + oreg]$
else if $(breg + oreg) < (ab + as)$ **then** $breg \leftarrow mem[ab + breg + oreg]$
else error

Solution: In the lectures the students saw an example for a store instruction. This question tests the application to a load instruction.

The correct definition should only allow the unrestricted load if *ink* == 1.

Otherwise, offsets $oreg$ are calculated from zero (to ensure relocatable programs). Therefore the resulting memory address $breg + oreg$ should be lower than the highest allowed address size as , relative to zero. The correct comparison is $(breg + oreg) < as$.

The final memory access is offset by the application base ab to translate from the relocatable zero to the real position in memory.

Each incorrect answer makes the wrong comparison and/or offset calculation.

41. In adding protection to the Hex 8 ISA, we must ensure that only the kernel can access all memory addresses. To do this, we add a flag ink which is 1 if the processor is executing instructions associated with the kernel; or 0 otherwise. We also add two registers: ab , the application base memory address; and as , the application size.

The LDAP instruction was originally defined as follows:

LDAP $areg \leftarrow pc + oreg$

Which of the following is the correct definition of a *protected* LDAP Hex 8 instruction?

- A. if ink is 1 then $areg \leftarrow pc + oreg$
else if $(pc + oreg) < as$ then $areg \leftarrow ab + pc + oreg$
else error
- B. if ink is 0 then $areg \leftarrow pc + oreg$
else if $(pc + oreg) < as$ then $areg \leftarrow ab + pc + oreg$
else error
- C. if ink is 1 then $areg \leftarrow pc + oreg$
else if $(pc + oreg) < as$ then $areg \leftarrow as + pc + oreg$
else error
- D. if ink is 0 then $areg \leftarrow pc + oreg$
else if $(pc + oreg) < as$ then $areg \leftarrow as + pc + oreg$
else error
- E. $areg \leftarrow pc + oreg$
i.e., no changes required.

Solution: In the lectures the students saw an example for a store instruction. This question tests understanding of when protection is required.

This instruction uses the pc , which will be set to a value within the valid range the program can access. The instruction sets the $areg$ to the sum of the pc and $oreg$, but does not try to access this address in memory. If this sum would take the value out of the valid range, the error would occur on an instruction trying to set the pc or access memory at this address, and *not* this instruction. Therefore, this instruction does not need modifying.

The other (incorrect) versions apply the transformation shows in the lecture to check the application base in various ways.

42. What mechanism do operating system kernels use to schedule programs?

- A. Subroutine call
- B. Pipelining
- C. Interrupts**
- D. Exceptions
- E. Memory protection

Solution: Interrupts (hardware and software) are used to return the execution into the kernel giving it an opportunity to schedule programs.

The other answers are spurious.

43. Many programming languages support recursive subroutine calls; specifically, a subroutine is allowed to call itself. Hardware and software interact to make subroutine calls efficient.

Which of the following features puts a limit on the number of subroutine calls that a program can make on a given architecture?

You may assume the registers are wide enough to address any word in memory.

- A. The stack pointer register.
- B. The return address register.
- C. The number of architectural registers.
- D. The size of the stack.**
- E. The size of the instruction cache.

Solution: This question asks to think about what resource would expire if there was infinite recursion.

As each subroutine call will push data onto a stack, for a very large number of calls, the stack memory would run out. This puts a limit on the number of subroutine calls, as beyond a certain point, no more calls could be supported without having to overwrite memory.

The registers are updated for each subroutine call and are independent on the number of subroutine calls made; their values are stored on the stack.

The instruction cache stores spatially close instructions, so will help the performance of a (small) recursive subroutine call.

44. A program which is position independent is loaded into memory by the loader. The program is placed so that it begins at memory address P.

Which of the following classes of instructions in a Reduced Instruction Set Computer (RISC) architecture, such as the Hex8 Instruction Set Architecture (ISA), might the loader need to update, given the programs location in memory is now known?

- A. Arithmetic operations.
- B. Relative branch instructions.
- C. System or kernel calls.
- D. Logical instructions.
- E. Memory load and store instructions.**

Solution: Mainly a fact check of information synthesised from lecture material. The lectures show how a store instruction might need to be modified given an application base, but the same procedure would apply to all memory access instructions.

Branch instructions will modify the program counter, which will be set to P when the program is loaded. Relative branches will not be affected by the programs position in memory; absolute branches would, but the choice above states “relative”.

45. A bus is used to communicate data between the CPU and memory, which operate on different clocks frequencies. A number of signal/control wires are used to implement a communication protocol between the CPU and memory.

The CPU requests data from memory by sending the appropriate signals for a read. How does CPU know that the data has arrived and it is safe to copy from the bus?

- A. The data has arrived after the memory signals it is ready.**
- B. The data has arrived after a fixed number of CPU cycles.
- C. The data has arrived after a fixed number of memory cycles.
- D. The data has arrived after the CPU signals it is ready.
- E. The data arrives after a fixed number of both CPU and memory cycles.

Solution: This question tests the different situations where synchronous and asynchronous buses are used, and how they work at a high level. The key detail is that the CPU and memory have different clocks, and so an *asynchronous* bus must be used, which operates on control signals rather than against the clock. Therefore, the data arrives once the memory tells the CPU it's on the bus.

46. A program has a bug. During execution, an instruction is issued which attempted integer division by zero. The Arithmetic Logic Unit stores this information in the status register.

What type of error is this known as?

- A. Asynchronous Interrupt.
- B. Synchronous Exception.**
- C. Asynchronous Error.
- D. Overflow.
- E. Carry.

Solution: Because the error occurred as a result of instruction execution, the error is a synchronous exception.

An interrupt happens asynchronously, and externally from the processor, so this is incorrect. Divide by zero is not a carry or overflow.

47. One of my programs has a bug, and I want to use a debugger to try to find where in the program the issue occurs. I set a breakpoint to stop execution of my program just before a subroutine call so I can investigate the program state interactively.

What mechanism could the processor provide to support this?

- A. Timing interrupts.
- B. Software interrupts.**
- C. Hardware interrupts.
- D. A system call to enable printing to the screen.
- E. Exceptions.

Solution: The question hints I want to stop the program at a specific point in the program. Debuggers usually set breakpoints by replacing instructions with software interrupts.

A timing interrupt would stop the program after a given time, which is not necessarily at a specific point in the program source.

A debugger must interact with the kernel (via software interrupts), but simply providing `printf()` support alone would not let me explore the state interactively.

Exceptions would create errors in the program, which could be caught by a debugger, but again, it doesn't stop at a specific point (a subroutine) in the program source.

48. A keyboard is used to input text characters into a processor. When a key is pressed, the keyboard sends an 8-bit signal to the system. An interrupt handler is used to copy that signal into a buffer in memory, storing it as a text character. The interrupt handler also updates a counter which contains the number of characters currently in the buffer.

You are writing a print program that displays the characters entered on the keyboard. In other words, your program prints the contents of the memory buffer to the screen, clears the memory buffer, and then the program ends.

The program is intended to be run by a privileged user, with full access to the entire memory space.

In order to ensure correct operation, why should interrupts be disabled when the print program is running.

- A. The interrupt handler would miss future keyboard presses while the print program is running.
- B. The interrupt handler needs to restore the state of the print program after running.

- C. The interrupt handler and the printing program could change the value stored in the counter at the same time.**
- D. The print program cannot change the value in registers.
- E. The print program cannot change the data in memory.

Solution: If interrupts were not disabled, the interrupt handler could try to increment the counter whilst the print program is reading/writing this value. In addition, the counter is used to show the offset in memory to append new characters to the buffer; and so data can be lost if the interrupt handler ran and overwrote data in the buffer. Therefore, interrupts should be disabled to prevent this memory race.

The print program can of course change the data in memory (the question says it will), and it will use registers. The statements about the interrupt handler are true, however are not the reasons why interrupts should be disabled.

49. The x86 Instruction Set Architecture (ISA) has a HLT (halt) instruction. This instruction takes no operands and is only available under privileged execution (by the kernel). The effect of the instruction is to stop the execution of the processor, i.e., no further instructions will be executed.

How could the processor recover from this instruction and continue executing the kernel, maintaining the architectural state of the processor (registers and memory)?

- A. The processor cannot be restarted.
- B. Reset the power source.
- C. Resume after an external interrupt occurs.**
- D. Resume after a software interrupt occurs.
- E. Resume executing a non-privileged program.

Solution: The HLT instruction stops execution until an interrupt occurs.

As the processor has ceased execution, it is unable to process further instructions (like software interrupts, or a non-privileged program). Rebooting the processor would cause the state of registers and (DRAM) memory to be erased.

6 Assembly and Assemblers

50. Consider the following instructions:

XADD r1, r2 $temp \leftarrow r1 + r2; r2 \leftarrow r1; r1 \leftarrow temp$

ADD r1, r2 $r1 \leftarrow r1 + r2$; set the carry flag (CF) if the result overflowed.

which are inspired by the x86 ISA, and operate on 4-bit registers. If we set $r1 = 0$ and $r2 = 1$, then execute the following program

```
xadd r1, r2
add  r1, r2
xadd r1, r2
add  r1, r2
xadd r1, r2
add  r1, r2
xadd r1, r2
...
```

what sequence of values does the register $r1$ take?

- A. $r1 = 0, 1, 1, 2, 3, 5, 8, 13, \dots$
- B. $r1 = 0, 1, 1, 1, 2, 3, 5, 7, \dots$**
- C. $r1 = 0, 1, 1, 1, 2, 2, 5, 8, \dots$
- D. $r1 = 1, 0, 0, 1, 1, 2, 2, 5, \dots$
- E. $r1 = 1, 0, 0, 1, 1, 3, 3, 5, \dots$

Solution: The XADD instruction computes the sum of the registers, storing the source $r1$ in $r2$ and storing the result in $r1$. The ADD instruction computes the sum of the registers, storing the result in $r1$, leaving $r2$ unmodified.

The registers therefore take the following values:

r1	r2	
0	1	
1	0	xadd
1	0	add
1	1	xadd
2	1	add
3	2	xadd
5	2	add
7	5	xadd
...		

51. During the assembly of an assembly program, labels are resolved. Consider the following Hex 8 assembly program:

```
Line 1 -   ldbc loop
Line 2 -   loop: ldac 0
Line 3 -   add
Line 4 -   br loop
```

During label resolution, what constant offsets are the operands on Line 1 (loop) and Line 4 (loop) replaced with?

- A. Line 1 becomes br -1 and line 4 becomes ldbc -1
- B. Line 1 becomes br 4 and line 4 becomes ldbc 4
- C. Line 1 becomes br 0 and line 4 becomes ldbc 4
- D. Line 1 becomes br 0 and line 4 becomes ldbc 0
- E. Line 1 becomes ldbc 2 and line 4 becomes br -3**

Solution: After label resolution the program will be:

```
Line 1 -   ldbc 2
Line 2 -   ldac 0
Line 3 -   add
Line 4 -   br -3
```

The Hex 8 processor has an execution cycle of Fetch/Increment PC/Execute. The BR instruction computes $pc \leftarrow pc + oreg$. Just before line 4 BR is executed $pc = 5$. Therefore to set $pc = 2$, the operand must be -3 .

On line 1, the LDBC instruction sets $breg \leftarrow oreg$. With an operand of the label loop, this instruction needs the address of the label. Therefore the operand must be 2.

52. During the assembly of an assembly program into machine code, labels are resolved. Consider the following Hex 8 assembly program:

```
Line 1 - BR L1
Line 2 - L2
Line 3 - DATA 1
Line 4 - L3
Line 5 - DATA 1
Line 6 - L1
Line 7 - LDBM L2
Line 8 - LDAM L3
Line 9 - STAM L2
```

```

Line 10 - ADD
Line 11 - STAM L3
Line 12 - LDBC 8
Line 13 - SUB
Line 14 - BRN L1

```

During label resolution, the labels on lines 7, 8, 9, 11 and 14 are resolved. What constant values are they replaced with, respectively? Assume no extra instructions are created.

Recall the Hex 8 execution cycle: Fetch, Increment program counter (*PC*), Execute. You may assume that the *PC* is set to the line number when that line is fetched.

- A. 2, 4, 2, 4, -8
- B. 3, 5, 3, 5, 8
- C. -4, -3, -6, -6, -8
- D. 2, 4, 2, 4, 8
- E. 3, 5, 3, 5, -8**

Solution: This Hex 8 program generates the Fibonacci numbers in memory up to 8.

L1 refers to the instruction on Line 7. L2 refers to the instruction on Line 3. L3 refers to the instruction on Line 5.

The LDAM, LDBM, and STAM instructions all take immediate values for the label. Hence, L2 = 3 on lines 7 and 9, and L3 = 5 on lines 8 and 11.

The BRN instruction updates the *PC*, and so the relative offset must be used. The *PC* is incremented after fetch, and before execute, so the *PC* is one greater than the line number. Hence, L1 resolves to $7 - 15 = -8$ on Line 14.

53. During the assembly of an assembly program into machine code, labels are resolved. Consider the following Hex 8 assembly program:

```

Line 1 - L1
Line 2 - DATA 15
Line 3 - L2
Line 4 - LDBC 1
Line 5 - LDAM L1
Line 6 - SUB
Line 7 - STAM L1
Line 8 - BRZ L3
Line 9 - BR L2
Line 10 - L3
Line 11 - LDAC 15
Line 12 - STAM L1

```


During label resolution, the labels L1, L2 and L3 on lines 5, 7, 8, 9 and 12 are resolved. What constant values are they replaced with, respectively? Assume no extra instructions are created.

Recall the Hex 8 execution cycle: Fetch, Increment program counter PC , Execute. You may assume that the PC is set to the line number when that line is fetched.

- A. -4, -6, 2, -6, -11.
- B. -4, -6, 3, -5, -11.
- C. 2, 2, 3, -5, 2.
- D. 2, 2, 4, 11, 2.
- E. 2, 2, 2, -6, 2.**

Solution: The Hex 8 program counts from 15 to zero.

L1 refers to the instruction on Line 2. L2 refers to the instruction on Line 4. L3 refers to the instruction on Line 11.

The LDAM and STAM instructions all take immediate values for the operand, and so the labels take their absolute value. Hence, L1 on lines 5, 7 and 12 all resolve to 2.

The branch instructions BRZ and BR update the PC , and so a relative offset must be used. The PC is incremented after fetch, and before execute, so the PC is one greater than the line number. On Line 8, $PC = 9$, and so L3 resolves to $11 - 9 = 2$.

On Line 9, $PC = 10$, and so L2 resolves to $4 - 10 = -6$.

54. During the assembly of an assembly program into machine code, labels are resolved. Consider the following Hex 8 assembly program:

```
Line 1  - BR L1
Line 2  - L2
Line 3  - DATA 1
Line 4  - DATA 2
Line 5  - DATA 3
Line 6  - DATA 4
Line 7  - DATA 5
Line 8  - L3
Line 9  - DATA 0
Line 10 - L1
Line 11 - LDAC L3
Line 12 - LDAI L2
Line 13 - LDBM L3
Line 14 - ADD
Line 15 - LDBI L2
Line 16 - STAI L3
```

During label resolution, the labels on lines 1, 11, 12, 13, 15 and 16 are resolved. What constant values are they replaced with, respectively? Assume no extra instructions are created.

Recall the Hex 8 execution cycle: Fetch, Increment program counter (*PC*), Execute. You may assume that the *PC* is set to the line number when that line is fetched.

- A. 11, 9, 3, 9, 3, 9
- B. 9, 1, 8, 3, 11, 6
- C. 9, 9, 3, 9, 3, 9**
- D. 11, 3, 9, 3, 9, 3
- E. 9, 9, 9, 9, 9, 9

Solution: This Hex 8 program uses indirect memory accesses.

L1 refers to the instruction on Line 11. L2 refers to the instruction (data) on Line 3. L3 refers to the instruction (data) on Line 9.

The LDAC, LDAI, LDBM LDBI and STAI instructions all take immediate values for the label. Hence, L2 = 3 on lines 12 and 15, and L3 = 9 on lines 11, 13 and 16.

The BR instruction updates the *PC*, and so the relative offset must be used. The *PC* is incremented after fetch, and before execute, so the *PC* is one greater than the line number. Hence, L1 resolves to $11 - 1 - 1 = 9$ on Line 1.

7 Compilers

55. Which of these tasks is *not* performed by a compiler?

- A. Optimisation
- B. Parsing
- C. Machine code output
- D. Application base memory location**
- E. Linking

Solution: The kernel is responsible for placing the compiled program in memory thus determining its base memory location.

All the others are functions of the compiler toolchain.

56. Explain the following compiler optimisations:

1. Common subexpression elimination
2. Dead code elimination
3. Constant folding
4. Hoist loop invariants
5. Function inlining
6. Loop induction variables
7. Strength reduction

Solution: These are defined in the lecture slides.

57. Consider the following extract from a C program:

```
int i, j;
int A[80];
for (i = 0; i < 8; ++i) {
    for (j = 0; j < 10; ++j) {
        int k = i * 10 + j;
        A[k] = j * 10 + i;
    }
}
```

Apply the strength reduction optimisation to this program.

How many additions and multiplications now occur in total, *excluding* the increments to *i* and *j* in the for loops?

- A. 116 additions and zero multiplications.
- B. 28 additions and zero multiplications.
- C. 168 additions and zero multiplications.**
- D. 28 additions and 2 multiplications.
- E. 256 additions and zero multiplications.

Solution: It has a number of steps, including counting at the end to answer the question directly.

There are two loops, so we apply the optimisation twice. For the inner *j* loop, the result of the optimisation is:

```
int i, j;
int A[80];
for (i = 0; i < 8; ++i) {
    int l = i * 10 + 0;
    int m = 0 * 10 + i;
    for (j = 0; j < 10; ++j) {
        A[l] = m;
        l = l + 1;
        m = m + 10;
    }
}
```

Now, applying it to the *i* loop. Note that in the above $m == i$, so we don't need to do anything to it, and the optimisation is only applied to *l*.

```
int i, j;
int A[80];
int p = 0;
for (i = 0; i < 8; ++i) {
    int l = p;
    int m = i;
    for (j = 0; j < 10; ++j) {
        A[l] = m;
        l = l + 1;
        m = m + 10;
    }
    p = p + 10;
}
```

There are therefore 10×2 additions in the *j* loop. Each iteration of the *i* loop performs the 20 additions for the *j* loop and 1 more, for a total of $8 \times 21 = 168$ additions, and zero multiplications.

58. Consider the following extract from a C program:

```
int i;  
int j = 3;  
int A[16];  
for (i = 0; i < 16; ++i) {  
    A[i] = i*16 + j;  
}
```

Apply the strength reduction optimisation to this program.

How many additions and multiplications now occur in total, *excluding* the increment to *i* in the for loop?

- A. 16 additions and one multiplications.
- B. 32 additions and one multiplications.
- C. 16 additions and zero multiplications.**
- D. 32 additions and zero multiplications.
- E. 16 additions and 16 multiplications.

Solution:

The result of the optimisation is:

```
int i;  
int j = 3;  
int A[16];  
int k = j;  
for (i = 0; i < 16; ++i) {  
    A[i] = k;  
    k = k + 16;  
}
```

There is therefore only one addition in the loop. There are 16 iterations of the loop, so the answer is 16 additions and zero multiplications.

59. Consider the following extract from a C program:

```
int function(int x) {  
    const int N = 12;  
    int a = 2 * N + x;  
    int b = 2 * N + x + 1;  
    int c = 3 * N;  
    int d = c + a;  
}
```

```
    return a + b;
}
```

An optimising C compiler might apply a number of transformations to this program to reduce the cost to execute it.

Without optimisation, the function would perform 3 multiplications and 5 addition operations.

Apply the following optimisations to the above program.

- Common subexpression elimination.
- Constant folding.
- Dead code elimination.

After applying the optimisations, in the order given, how many multiplications and addition operations must be performed?

- A. 3 multiplications and 5 additions.
- B. 3 multiplications and 4 additions.
- C. 1 multiplications and 3 additions.
- D. 0 multiplications and 3 additions.**
- E. 0 multiplications and 4 additions.

Solution:

After application of common subexpression elimination, the program would look as follows:

```
int function(int x) {
    const int N = 12;
    int a = 2 * N + x;
    int b = a + 1;
    int c = 3 * N;
    int d = c + a;
    return a + b;
}
```

After constant folding, the program would look as follows:

```
int function(int x) {
    const int N = 12;
    int a = 24 + x;
    int b = a + 1;
    int c = 36;
```

```
    int d = c + a;
    return a + b;
}
```

Finally, after dead code elimination, the program would look as follows:

```
int function(int x) {
    int a = 24 + x;
    int b = a + 1;
    return a + b;
}
```

Therefore, there are 3 additions, and 0 multiplications.

Further optimisation is possible, but these would not reduce the arithmetic beyond this minimum. It therefore does not really matter is the solution is obtained by some other means.

60. Consider the following extract from a C program:

```
const int N = 25;
int A[25];
int B[25];
for (int i = 0; i < N; ++i) {
    A[i] = i + 12;
    int j = i + N;
    int k = i + 12;
    B[i] = i + 12 + N;
}
```

An optimising C compiler might apply a number of transformations to this program to reduce the cost to execute it.

Without optimisation, the function would perform $25 \times 6 = 150$ addition operations.

Apply the following optimisations to the above program.

- Common subexpression elimination.
- Constant folding.
- Dead code elimination.

After applying the optimisations, in the order given, how many addition operations must be performed?

A. 2

- B. 25
- C. 50
- D. 75**
- E. 100

Solution: Application of a procedure from the lectures. It has a number of steps, including counting at the end to answer the question directly.

After application of common subexpression elimination, the program would look as follows:

```
const int N = 25;
int A[25];
int B[25];
for (int i = 0; i < N; ++i) {
    int a = i + 12;
    A[i] = a;
    int j = i + N;
    int k = a;
    B[i] = a + N;
}
```

After constant folding, the program would look as follows:

```
const int N = 25;
int A[25];
int B[25];
for (int i = 0; i < N; ++i) {
    int a = i + 12;
    A[i] = a;
    int j = i + 25;
    int k = a;
    B[i] = a + 25;
}
```

Note that because common subexpression was performed first, the final line still contains a + operation after constant folding.

Finally, after dead code elimination, the program would look as follows:

```
const int N = 25;
int A[25];
int B[25];
for (int i = 0; i < N; ++i) {
```



```
int a = i + 12;
A[i] = a;
B[i] = a + 25;
}
```

Therefore, there are 2 additions in the loop body and one to increment *i* each iteration. There are 25 iterations, therefore $25 \times 3 = 75$ additions.

61. A compiler is used to translate programs written in a high-level programming language to machine code (*MC*). When a new programming language is developed, a compiler must be written to support this new language. This process is often known as bootstrapping.

I have developed a new programming language called *Blue*, and implement a compiler in my second-favourite programming language, *C*, so that I can compile a program written in *Blue*. I then implement a *Blue* compiler in the *Blue* language. Which of the Tombstone diagrams (T-diagrams) shown in Figure 2 on correctly represents this bootstrapping process for my *Blue* compiler? You may assume a *C* compiler already exists.

- A.
- B.
- C.
- D.
- E.

Solution: Each Tombstone (T diagram) shows a compiler. There are two compilers in the question:

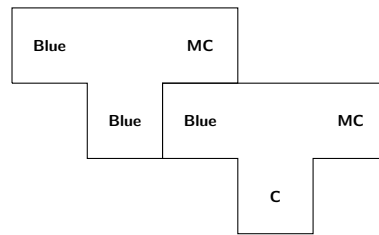
1. A *Blue* to *MC* compiler, written in *Blue*.
2. A *Blue* to *MC* compiler, written in *C*.

Therefore, the first compiler can be built with the second, giving a path from *Blue* to *MC*.

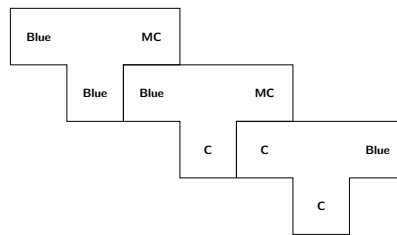
The option B shows a third *C* compiler, which the question assumes to already exist, so does not need to be shown. In addition, the question does not state what this *C* compiler is written in. To make this clear, the potential answer shows this is implemented in *Blue*, which is not correct.

The other options jumble the location of the languages to produce incorrect diagrams.

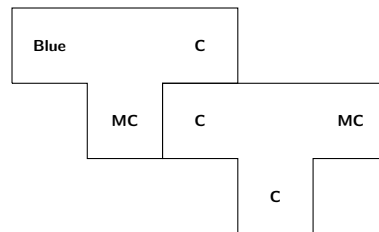
A



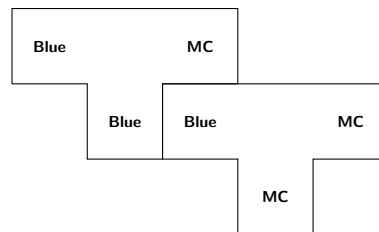
B



C



D



E

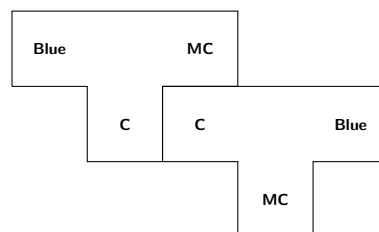


Figure 2: Tombstone diagrams (T-diagrams) for bootstrapping a *Blue* compiler